JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Generative AI

Audio and Music Generation

Outline

- Module Learning Objectives
- WaveNet
- RNN and LSTM review
- Audio quality evaluation framework
- MuseGAN

Module Learning Objectives

- Define and outline WaveNet neutral network architecture
- Review RNN, LSTM methods
- Demonstrate an audio quality evaluation framework using MFCC and perceptual methods
- Analyze and evaluate MuseGAN

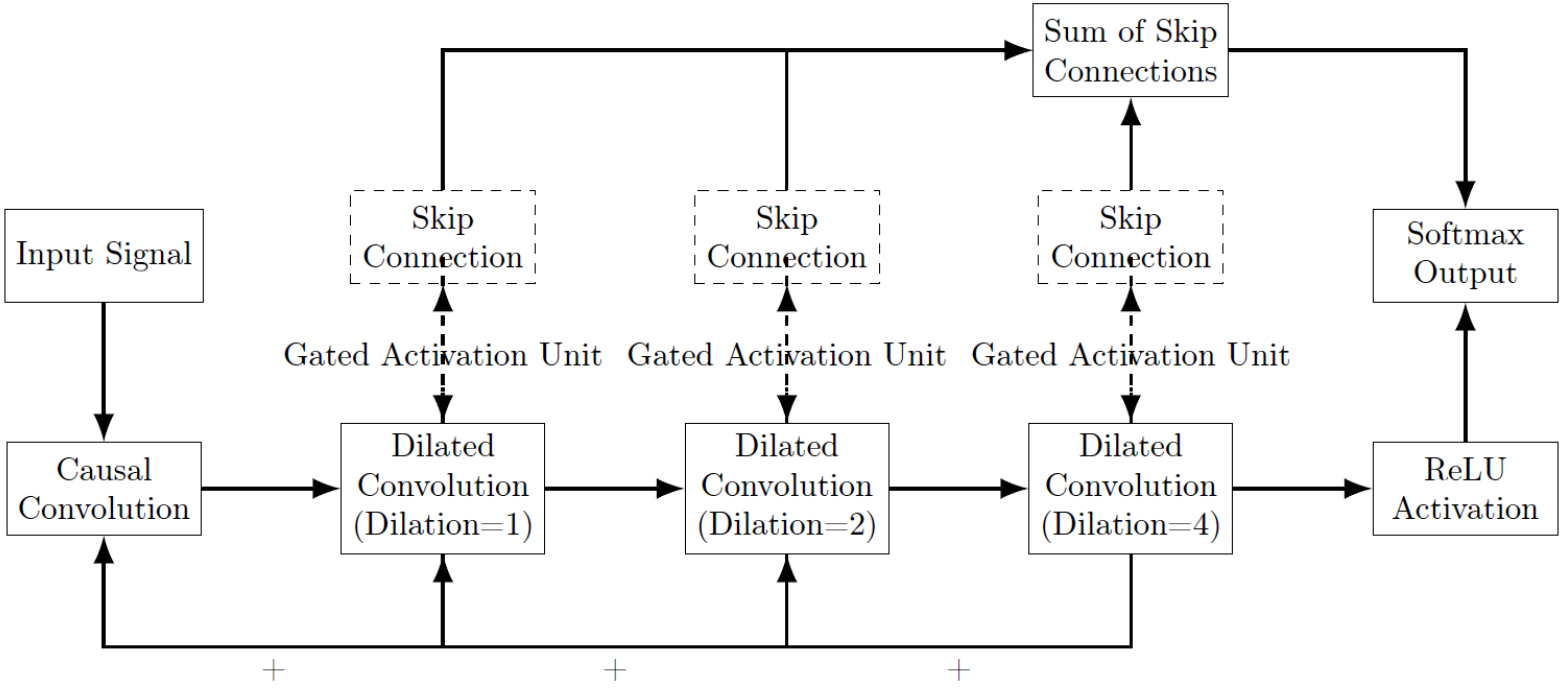
Introduction

- Music and audio generation focus on creating sound, melodies, harmonies, and rhythms that emulate human-composed music or unique, AI-driven compositions
  - Intricate patterns, emotional appeal, and genre-specific structures
- Generative AI expands creative potential by assisting musicians and composers
  - Personalized playlists, adaptive soundtracks in gaming, virtual environments, etc.
- **Sequence Models** treats music as a time-sequenced data stream
  - Recurrent neural networks and transformers are suitable models
- **Image-Based Models** treats music as a two-dimensional image (pitch vs. time) allowing models like MuseGAN to generate entire tracks or styles
  - Allowing control of components (chords, style, melody, groove) independently
- **GANs** are especially useful for polyphonic music generation, and they can create complex, multi-layered harmonies by simulating competing voices and styles

WaveNet Neutral Network

- WaveNet is a deep generative model by DeepMind (2016) producing raw audio
- Utilizes a convolutional neural network (CNN) architecture with dilated causal convolutions to model the conditional probabilities of audio samples
- WaveNet is capable of generating high-fidelity audio, including human-like speech and music, by learning complex temporal dependencies in audio data
- **Causal Convolutions** ensure that the model respects the temporal order of data
- **Dilated Convolutions** introduce gaps between the filter taps, allowing the network to have a large receptive field without increasing computational cost
- **Residual and Skip Connections** allow gradients to flow through the network effectively for training deep networks

WaveNet Architecture



WaveNet Mathematical Model

- Given a raw audio waveform $X = \langle x_1 x_2 \dots x_T \rangle$, WaveNet models the joint probability

$$P(x) = \prod_{t=1}^{T} P(x_t | x_1, x_2, \dots, x_{t-1}) = \mathrm{softmax}(W \star h_t)$$

- Causal convolutions ensure the predictions for each t only depend on inputs from time steps $\leq t$

$$y_t = \sum_{i=0}^{K-1} f_i \cdot x_{t-i}$$

- Dilated convolutions capture long-range dependencies where d controls the gap

$$y_t = \sum_{i=0}^{K-1} f_i \cdot x_{t-d \cdot i}$$

Mathematical Model

- Dilated Causal Convolution

$$(x \star df)(t) = \sum_{k=0}^{K-1} f(k)x(t - d \cdot k)$$

  - where d is the dilation factor, K is the filter size and $f(k)$ are filter weights
  - d controls the spacing between filter elements, allowing the network to capture wider temporal dependencies without increasing the number of parameters
- Gated Activation Unit increase expressiveness

$$z(t) = \tanh(W_f \star dx(t)) \odot \sigma(W_g \star dx(t))$$

  - W_f and W_g are filter and gate weights; $\odot$ denotes element-wise multiplication

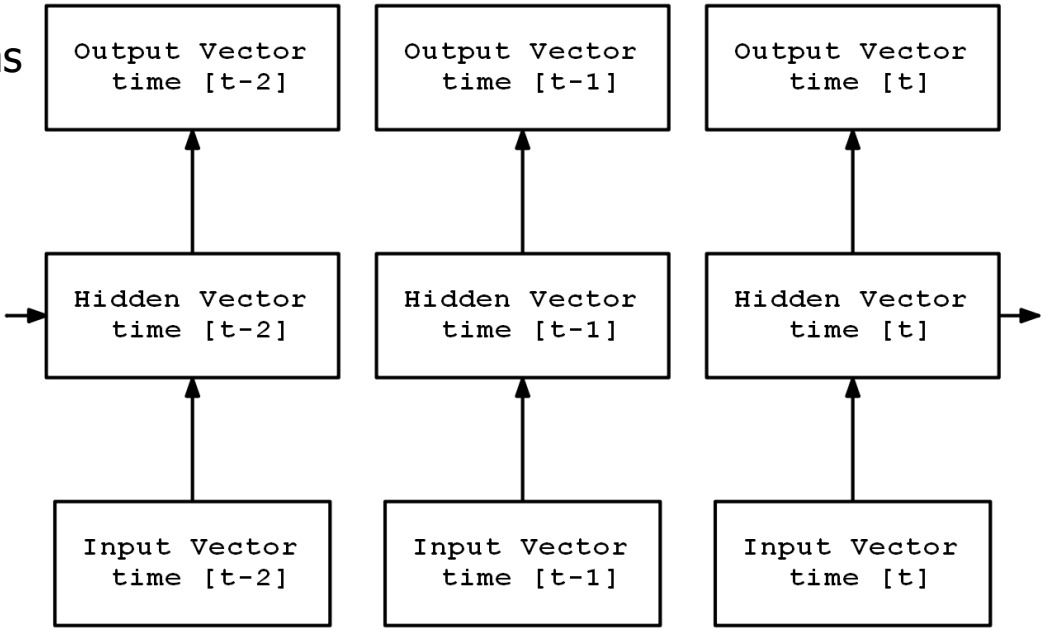
Mathematical Model

- Residual and Skip Connections allow the model to stack many layers without vanishing gradients, enhancing training stability
  - Residual output: $r(t) = x(t) + W_r \star z(t)$
  - Skip output: $s(t) = W_s \star z(t)$
  - where W_r and W_s are learned weights
- Softmax Output for Audio Prediction produces a distribution over discrete audio sample values (often quantized into 256 values with μ-law encoding)

$$P(x_t \mid x_{1:t-1}) = \mathrm{softmax}(W \star h)$$

Recurrent Neural Networks

- A class of NN designed to recognize patterns in sequences of data such as text, genomes, handwriting, or the temporal dynamics of audio signals
- In the context of audio/music generation, RNNs are employed to model temporal dependencies by maintaining a hidden state that captures information from previous time steps, allowing the network to generate coherent and contextually relevant audio sequences
  - Building block of a transformer



Long Short-Term Memory RNN

- In practice, RNNs are susceptible to the vanishing gradient problem, or exploding gradient due to the nature of the feedback network
- LSTM variant has three layers for control: input, output, forget gates

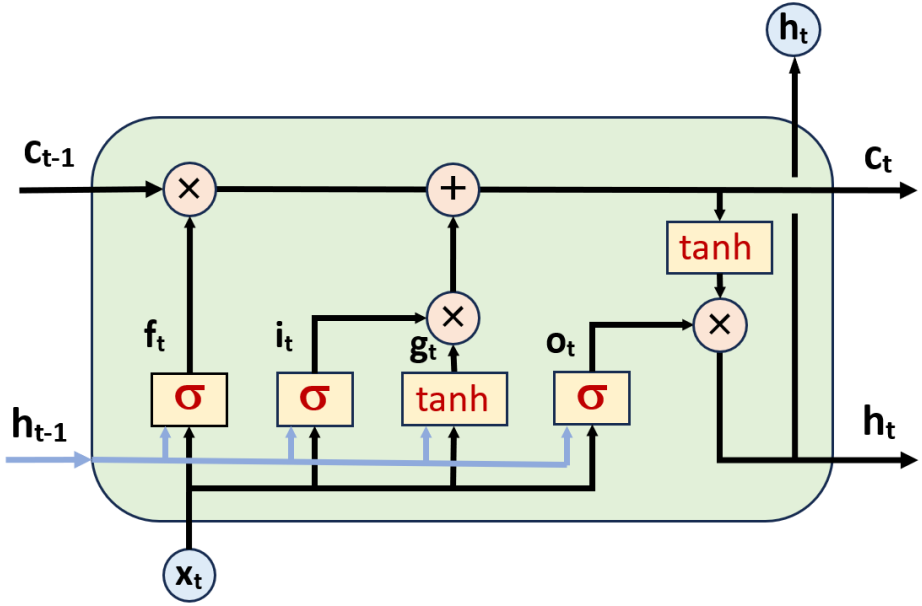
$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi})$$

$$g_t = \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg})$$

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})$$

$$f_t = \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf})$$

- Cell state $c_t = f_t \odot c_{t-1} + i_t \odot g_t$
- Hidden state $h_t = o_t \odot \tanh c_t$



Example RNN

```
class PyTorchRNN(nn.Module):
  def __init__(self, n_hidden=128, numH=2,
epochs=500, eta=0.005, B=1920, w=None,
dropout=0.0):
    super().__init__()
    self.rnn, self.outlayer = None, None
    self.softmax = nn.LogSoftmax(dim=1) #
loss function, last layer is nn.LogSoftmax
    self.loss_func = nn.NLLLoss(weight=w)

  def forward(self, _X):
    embeds = self.letter_embeddings(_X)
    lstm_out, _ = self.rnn(embeds) # output
is batched
    # Use last hid layer outp - W x Seq[T]
    output = self.outlayer(lstm_out[:, -1,
:]) # Use only the last hidden layer - Hn
    output = self.softmax(output)
    return output
```

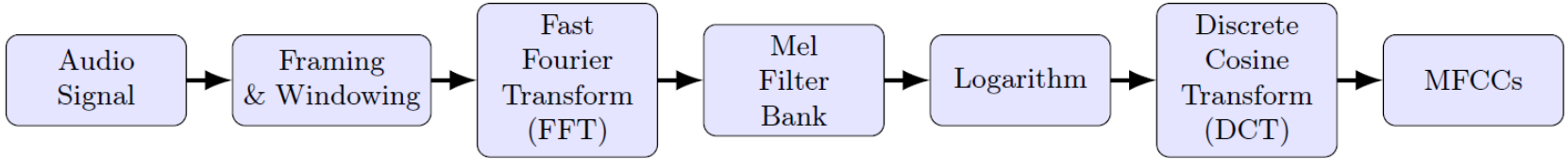
```
  def predict(self, _X):
    with torch.no_grad():
      output = self.forward(_X)
    p_values, indices = output.max(dim=1)
    return indices.to('cpu')

  def fit(self, _X_train, _y_train,
info=False, **kwargs):
    self.letter_embeddings =
nn.Embedding(M, EMBEDDING_M, device=Device)
    self.rnn = nn.LSTM(EMBEDDING_M,
self.n_hidden, self.numH,
bidirectional=False, bias=False,
batch_first=True, dropout=self.dropout,
device=Device)
    self.outlayer =
nn.Linear(self.n_hidden, C, device=Device)
    self.optimizer =
optim.Adam(self.parameters(), lr=self.eta)
```

Frequency Spectrum Features

- Recurrent neural networks can model sequences such as audio or text
- Audio or time-series data can also be modeled by frequency domain representations
- Mel-Frequency Cepstral Coefficients (**MFCCs**) are **features** and a representation of the short-term power spectrum of an audio signal, based on a linear cosine transform of a log power spectrum on a nonlinear mel scale of frequency
- Framing and Windowing dividing the signal into frames, applying windowing
- Fast Fourier Transform (FFT) converts the time-domain signal to frequency domain
- Mel Filter Bank applying a set of triangular filters spaced according to the mel scale
- Logarithm Computing the logarithm of the filter bank energies
- Discrete Cosine Transform (DCT) transforms the log energies to obtain the cepstral coefficients

Mel-Frequency Cepstral Coefficients



- Mel Scale relates perceived frequency to actual measured frequency

$$m(f) = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

- Fast Fourier Transform

$$S_p[k] = \sum_{n=0}^{N-1} s_p[n] \cdot e^{-\frac{j2\pi kn}{N}}, 0 \leq k < N$$

- Mel Filter Bank

$$E_k = \sum_{f=f_{min}}^{f_{max}} \left|S_p(f)\right|^2 \cdot H_k(f)$$

- Discrete Cosine Transform

$$c_n = \sum_{k=1}^{K} \log E_k \cdot \cos\left[\frac{\pi n(k-0.5)}{K}\right], 1 \leq n < L$$

Transformer-Based Music Generation

- Music notes are treated as tokens in a sequence, akin to words in a sentence
- Predict the next note based on prior notes, producing coherent musical output
- Given a sequence of musical notes, $X = \langle x_1 x_2 \dots x_{T-1} \rangle$, the transformer maximizes:

  $P(x_T | x_1, x_2, \dots, x_{T-1})$

- Attention mechanism of each note x_t is weighted by its relevance to the next note, where attention scores α_{ij} are computed as

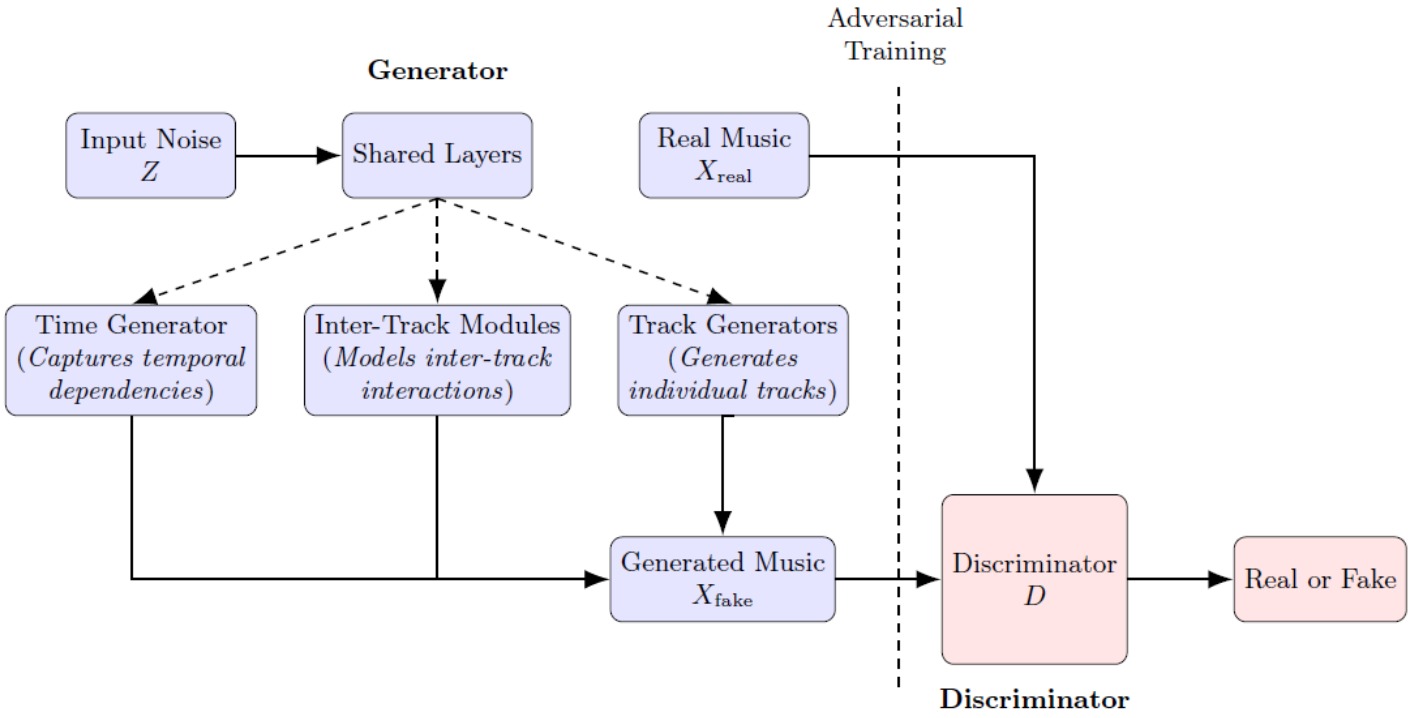
  $$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$$

- and $e_{ij} = \frac{(Q \cdot K^T)}{\sqrt{d_k}}$ for queries Q and keys K

MuseGAN-Based Music Generation

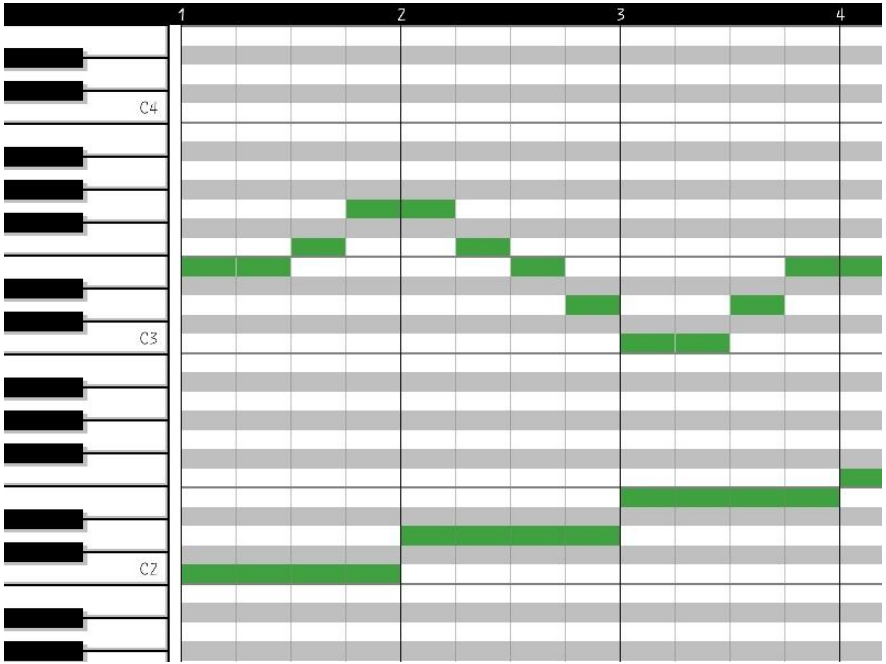
- MuseGAN is a GAN designed specifically for multi-track music generation
- Consists of a hierarchical architecture with multiple generators and discriminators organized to capture both temporal and inter-track dependencies
  - **Generator** generates multi-track music by producing piano-roll representations for each instrument over time
  - **Discriminator** evaluates the authenticity of the generated music by considering both individual tracks and their combinations
- **Track Generator** generates music for individual tracks
- **Time Generator** captures temporal dependencies across time steps
- **Inter-Track Modules** models the interactions between different instruments

MuseGAN Architecture



Pianoroll Representation of Music

- A pianoroll is a 2D visual representation of music where
  - The horizontal axis represents time (divided into time steps)
  - The vertical axis represents pitch (from low to high)
  - A cell at a given position (time, pitch) is activated if that note is played
- MuseGAN uses these pianoroll images



References

- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- Van Den Oord, Aaron, et al. "Wavenet: A generative model for raw audio." arXiv preprint arXiv:1609.03499 12 (2016).
- LSTM PtyTorch . https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html. Accessed Jul 2024.
- Dong, Hao-Wen, et al. "Musegan: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment." Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 32. No. 1. 2018.

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.